

OE-EC604B

Operating System

Combined Module Notes

Modules 1 – 9

Semester VI • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Biraj Sarkar

IN ASSOCIATION WITH

MAKAUT Wingman • MAKAUT Future Minds

Cooch Behar Government Engineering College

Exam-Ready Notes

Contents

Module 1: Operating System and Functions	5
1 Operating System and Functions	5
1.1 Conceptual View of a Computer System	5
1.2 Core Functions of an Operating System	5
2 Evolution of Operating Systems	6
2.1 Batch Processing Systems	6
2.2 Interactive & Multi-Programmed Systems	6
2.3 Time-Sharing Systems	6
3 Specialized Operating Systems	7
3.1 Real-Time Systems (RTS)	7
3.2 Multi-Threading Systems	7
Module 2: System Components and Operating System Services	11
1 System Components and Operating System Services	11
1.1 Operating System Services	11
1.2 System Calls	11
2 Operating System Architecture Structures	12
2.1 Monolithic Structure	12
2.2 Layered Structure	12
2.3 Microkernel Structure	12
2.4 Hybrid Systems	13
Module 3: Process Concept and Lifecycle States	17
1 Process Concept and Lifecycle States	17
1.1 Process Control Block (PCB)	17
1.2 Process States and Transitions	17
2 Process Generation & Scheduling Queues	17
2.1 Process Generation (<code>fork()</code> and <code>exec()</code>)	18
2.2 OS Schedulers	18
3 Inter-Process Communication (IPC)	18
4 Process Synchronization and Critical Section	18
4.1 The Critical Section Problem	18
4.2 Semaphores	19
4.3 Classical Synchronization Problems	19
4.3.1 Bounded-Buffer (Producer-Consumer) Problem	19
4.3.2 Readers-Writers Problem	19
4.3.3 Dining Philosophers Problem	19
4.4 Monitors	20

Module 4: Scheduling Concepts and Performance Criteria	24
1 Scheduling Concepts and Performance Criteria	24
1.1 CPU-I/O Burst Cycle	24
1.2 Scheduling Criteria	24
2 CPU Scheduling Algorithms	24
2.1 First-Come, First-Served (FCFS)	24
2.2 Shortest Job First (SJF)	25
2.3 Round Robin (RR) Scheduling	25
2.4 Priority Scheduling	25
2.5 Multilevel Queue (MLQ) & Multilevel Feedback Queue (MLFQ)	25
3 Multiprocessor Scheduling	26
Module 5: System Model and Deadlock Characterization	30
1 System Model and Deadlock Characterization	30
1.1 Four Necessary Conditions (Coffman Conditions)	30
1.2 Resource Allocation Graph (RAG)	30
2 Deadlock Handling Strategies	31
2.1 1. Deadlock Prevention	31
2.2 2. Deadlock Avoidance	31
2.2.1 Banker's Algorithm	32
2.3 3. Deadlock Detection & Recovery	32
Module 6: Contiguous Memory Allocation	36
1 Contiguous Memory Allocation	36
1.1 Memory Partitioning Schemes	36
2 Non-Contiguous Memory Allocation	36
2.1 Paging Architecture	36
2.2 Segmentation	37
3 Virtual Memory and Demand Paging	37
3.1 Demand Paging & Page Faults	37
3.2 Page Replacement Algorithms	37
3.3 Thrashing	38
Module 7: I/O Hardware and Organization of I/O Functions	42
1 I/O Hardware and Organization of I/O Functions	42
1.1 I/O Communication Methods	42
1.2 I/O Buffering	42
2 Disk Physical Structure and Access Performance	43
3 Disk Scheduling Algorithms	43
3.1 First-Come, First-Served (FCFS)	43

3.2	Shortest Seek Time First (SSTF)	43
3.3	SCAN (Elevator Algorithm)	43
3.4	C-SCAN (Circular SCAN)	44
3.5	LOOK & C-LOOK	44

Module 8: File and Directory Concepts 48

1	File and Directory Concepts	48
1.1	File Concepts & Access Mechanisms	48
1.2	Directory Structures	48
2	File System Implementation and Allocation Methods	49
2.1	1. Contiguous Allocation	49
2.2	2. Linked Allocation	49
2.3	3. Indexed Allocation	49
3	Free Space Management	50

Module 9: Protection & Security 54

1	Protection & Security	54
1.1	Access Matrix Model	54
2	Introduction to Distributed Operating Systems	54
3	Case Study: The UNIX Operating System	55
3.1	System Architecture	55
3.2	The UNIX File System (Inode Structure)	55
3.3	UNIX Process Management & Scheduling	55

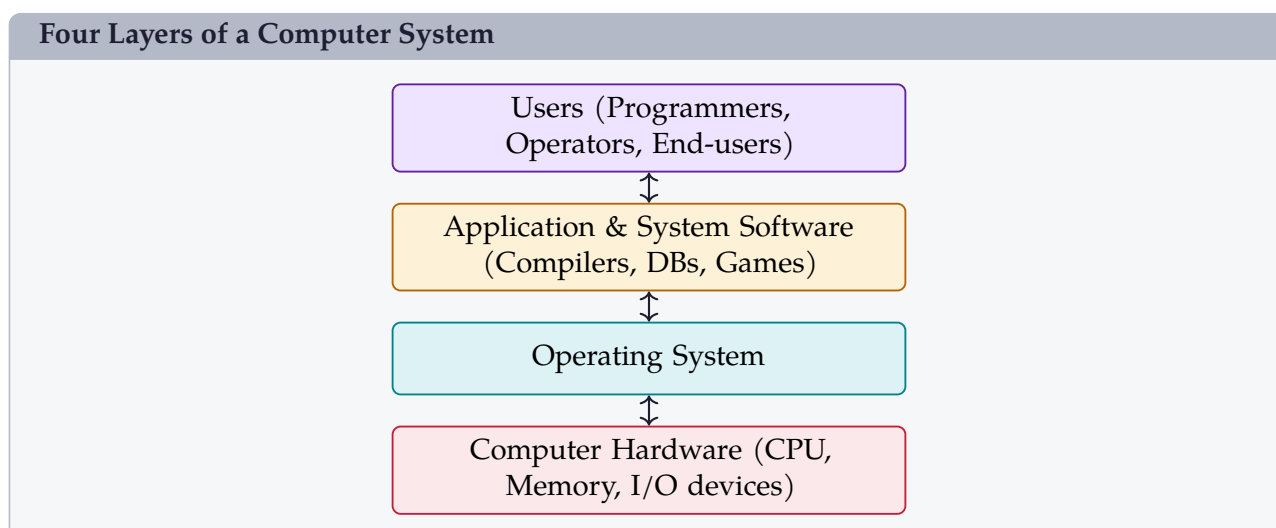
Module 1

Operating System and Functions

1. Operating System and Functions

An **Operating System (OS)** acts as an intermediary between the user of a computer and the computer hardware. Its primary goals are to make the computer system convenient to use and to execute user programs in an efficient manner.

1.1 Conceptual View of a Computer System



1.2 Core Functions of an Operating System

Definition: Operating System

An **Operating System** is a system software that manages computer hardware, software resources, and provides common services for computer programs.

From an engineering perspective, the functions of an OS can be classified into five major resource management categories:

Function	Description and Key Responsibilities
Process Management	Creation, execution, scheduling, synchronization, and termination of active processes.
Memory Management	Tracking free and allocated memory spaces, page mapping, swapping, and virtual memory allocation.
File System Management	Organising directories, files, mapping files to secondary storage, and access controls.
Device (I/O) Management	Buffering, caching, spooling, device drivers, and disk head scheduling.
Protection & Security	Controlling user access, authentication, preventing unauthorized access, and threat mitigation.

2. Evolution of Operating Systems

Operating systems have evolved from simple manual job setups on mainframes to complex distributed real-time systems.

2.1 Batch Processing Systems

Early mainframes used a batch system where the user had no direct access to the machine. Jobs were prepared offline on punched cards and grouped into batches by an operator.

- **Working Principle:** A program called the **resident monitor** stayed in memory, automatically loading and running subsequent jobs from the tape/card reader one after another.
- **Drawback:** No interactive user control, and extremely low CPU utilization due to the speed disparity between mechanical I/O devices (slow) and electronic CPU (fast).

2.2 Interactive & Multi-Programmed Systems

To overcome batch limitations, multi-programmed systems kept multiple jobs in memory simultaneously. When one job had to wait for I/O, the OS switched CPU execution to another job.

- **Interactive Systems:** Allowed direct communication between the user and system via a keyboard/terminal, giving immediate feedback.

2.3 Time-Sharing Systems

Time-sharing is a logical extension of multi-programming that provides fast interactive response times.

- **Working Principle:** The CPU executes multiple jobs by switching among them extremely fast using a fixed time quantum (slice). Each user gets a dedicated, albeit small, fraction of the CPU time, giving the illusion of a dedicated machine.
- **Key Goal:** Minimizing **response time** for multiple simultaneous users.

3. Specialized Operating Systems

3.1 Real-Time Systems (RTS)

A **Real-Time Operating System (RTOS)** is used when rigid time requirements are placed on the operation of a processor or the flow of data. The correctness of an RTOS depends not only on the logical result of computation but also on the **time** at which it is delivered.

Definition: Real-Time Systems

A **Real-Time System** guarantees that critical tasks are completed within strict, specified time constraints (deadlines).

Real-time systems are classified into two categories based on the severity of missing deadlines:

Feature	Hard Real-Time System	Soft Real-Time System
Deadline Violation	Catastrophic system failure.	Performance degradation only.
Tolerance	Zero tolerance. Deadlines must be met.	Occasional misses are tolerated.
Memory Support	Virtual memory is usually disabled.	Virtual memory is allowed.
Examples	pacemakers, missile guidance, airbags.	Video streaming, online gaming, audio.

3.2 Multi-Threading Systems

A **thread** (or lightweight process) is a basic unit of CPU utilization. In a multi-threading operating system, a single process can spawn multiple concurrent threads of execution, all sharing the same code segment, data segment, and system resources, but having their own thread ID, program counter, register set, and stack.

- **Benefits:** Responsiveness (non-blocking UI), resource sharing (threads share process memory), economy (context switching is faster than process creation), and multicore utilization.

Quick Revision — Module 1

Introduction to Operating Systems | OE-EC604B

Key Evolution Properties at a Glance

Batch OS	Low CPU utilization, high turnaround time, offline execution.
Multiprogramming	High CPU utilization, keeps multiple jobs in memory, switches on I/O.
Time-Sharing	Fixed time-slice (quantum), interactive, minimizes response time.
Hard Real-Time	Strict deadlines, safety-critical, no virtual memory, predictable delay.
Soft Real-Time	Best-effort deadlines, priority scheduling, virtual memory allowed.

One-Line Definitions

- **Operating System:** System software managing hardware resources and acting as a user-hardware interface.
- **Spooling:** Simultaneous Peripheral Operations On-Line; placing I/O data in disk buffer to speed up batch execution.
- **Multiprogramming:** OS technique running multiple jobs concurrently to maximize CPU utilization.
- **Time-Sharing:** Preemptive multiprogramming where CPU switches quickly between users using a time slice.
- **Real-Time OS:** Time-bound operating system designed for deterministic response times and deadlines.
- **Hard Real-Time:** OS where missing a single task deadline results in catastrophic system failure.
- **Soft Real-Time:** OS where deadlines are preferred but occasional misses do not cause system failure.
- **Thread:** A lightweight process representing a single sequential flow of control within a process.
- **Resident Monitor:** Early batch supervisor program residing in memory to load jobs automatically.

Frequently Asked / Viva Questions

Q1. What are the primary objectives of an operating system?

To make the computer system convenient to use for the user, and to allocate and

Frequently Asked / Viva Questions (contd.)

manage the hardware resources (CPU, memory, devices) in an efficient and fair manner.

Q2. Explain the difference between Multiprogramming and Multiprocessing.

Multiprogramming keeps multiple processes in main memory simultaneously on a single CPU, switching between them to keep the CPU busy. Multiprocessing involves a computer system with two or more physical CPUs executing processes in parallel.

Q3. What is spooling? How does it differ from buffering?

Spooling uses the disk as a large buffer to store incoming/outgoing jobs and data for slow peripheral devices (like printers), enabling concurrent CPU processing. Buffering is a temporary storage area in main memory used during I/O transfers to smooth out speed differences.

Q4. Differentiate between Hard Real-Time and Soft Real-Time systems.

Hard Real-Time systems have strict, non-negotiable deadlines where any failure causes catastrophic system collapse (e.g., flight controller). Soft Real-Time systems treat deadlines as highly desirable but tolerate occasional misses without failure (e.g., video player).

Q5. What is the time-sharing concept? Which scheduling is typically used?

Time-sharing allows multiple users to share a single CPU interactively. The OS allocates a small time slice (quantum) to each user process and switches preemptively using Round Robin scheduling to maintain low response times.

Q6. List the primary resource management functions of an OS.

Process management, Memory management, File system management, I/O device management, and Protection/Security management.

Q7. Why is virtual memory usually not supported in hard real-time systems?

Virtual memory introduces unpredictable time delays due to page faults and disk access latency, which violates the strict deterministic timing requirements of hard real-time systems.

Q8. Explain how multithreading improves application responsiveness.

By splitting execution into multiple threads, an application can execute background tasks (like disk writing or computations) in one thread while the main thread keeps responding to user actions (like button clicks) without freezing the UI.

Q9. What is the role of a resident monitor in early batch systems?

It was the first operating program. It resided in main memory and automatically handled job transitions by reading the next job card/tape as soon as the current one finished, reducing setup time.

Q10. Why is CPU scheduling required in multiprogrammed systems?

Frequently Asked / Viva Questions (contd.)

When the active process performs I/O, it must wait. To prevent the CPU from sitting idle, CPU scheduling selects another ready process from memory to execute, maximizing CPU utilization.

Comparative Analysis of Operating Systems

System Type	Primary Goal	CPU Utilization	Interaction Level
Batch	Maximize Throughput	Low to Moderate	None (offline jobs)
Multiprogramming	Maximize CPU usage	High (context switches on I/O)	Low (background jobs)
Time-Sharing	Minimize Response Time	Moderate to High	Very High (interactive)
Real-Time	Guarantee Deadline	Secondary concern	Low (automated control)

Module 2

System Components and Operating System Services

1. System Components and Operating System Services

An operating system provides an environment for the execution of programs by presenting services to both users and system processes.

1.1 Operating System Services

The OS provides services that make computer usage convenient and efficient:

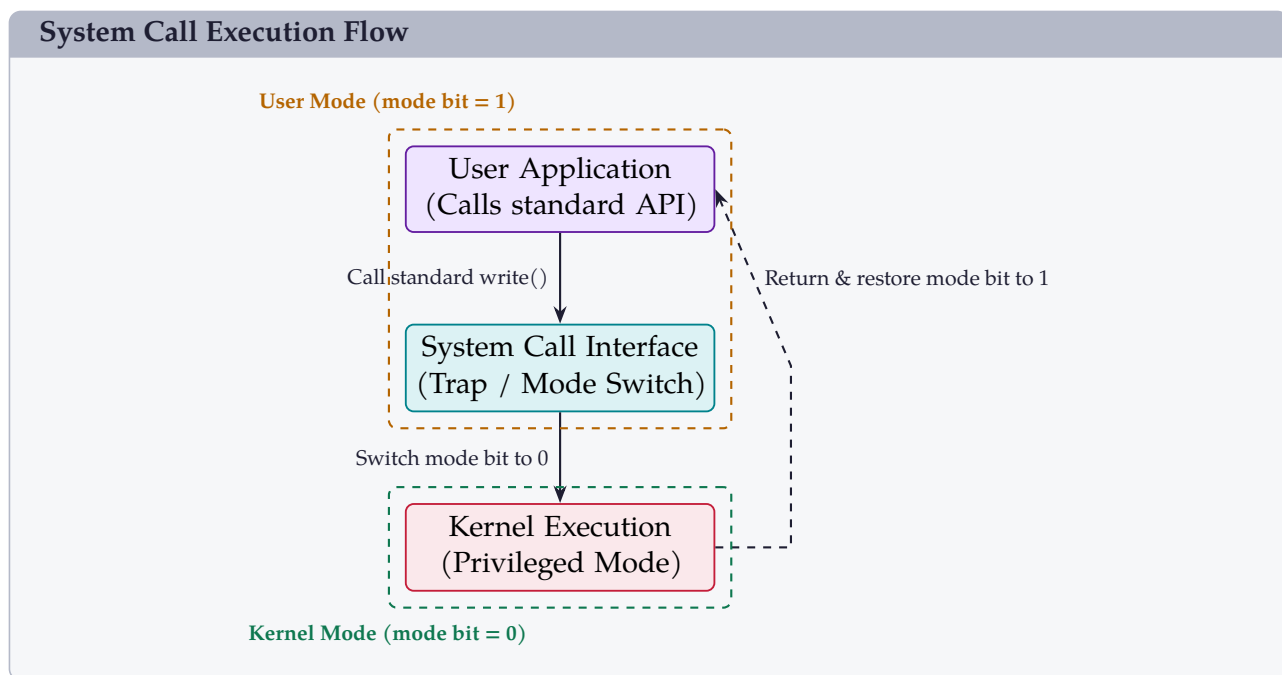
1. **User Interface (UI):** Command Line Interface (CLI), Graphics User Interface (GUI), or Batch Interface.
2. **Program Execution:** Loading a program into memory, executing it, and terminating it (either normally or abnormally).
3. **I/O Operations:** Managing read/write transfers to files or hardware devices since user programs cannot access hardware directly.
4. **File-System Manipulation:** Creating, deleting, searching, reading, writing, and controlling file access permissions.
5. **Communications:** Exchanging information between processes running on the same computer or different machines via networks (using shared memory or message passing).
6. **Error Detection:** Continuously monitoring for hardware errors (memory parity, power failure) or software errors (arithmetic overflow, illegal memory access) and executing handling procedures.
7. **Resource Allocation:** Distributing CPU cycles, main memory, and file storage dynamically among active users or jobs.
8. **Accounting:** Tracking user resource usage for billing, statistics, or system profiling.
9. **Protection & Security:** Controlling access to system resources and securing user authentication.

1.2 System Calls

Definition: System Call

A **System Call** is the programmatic interface that allows a running program (in user space) to request services directly from the operating system kernel (in kernel space).

- **Dual-Mode Operation:** Modern hardware supports at least two modes of operation: **User Mode (mode bit = 1)** and **Kernel/Supervisor Mode (mode bit = 0)**. System calls trigger a software interrupt (trap instruction) that shifts execution from user mode to kernel mode to run privileged instructions.



2. Operating System Architecture Structures

Operating systems must be structured to ensure reliability, performance, and ease of modification. There are four traditional design methodologies:

2.1 Monolithic Structure

In a monolithic operating system, all OS code (scheduling, filesystem, memory management, drivers) runs in a single address space in kernel mode.

- **Advantages:** High performance due to minimal context switching overhead.
- **Disadvantages:** Lack of safety — a bug in any module (like a camera driver) can crash the entire system.

2.2 Layered Structure

The OS is broken into a hierarchy of layers (0 to N). Layer 0 is the hardware; Layer N is the user interface.

- **Design Rule:** Each layer is built upon and can only invoke services of lower-level layers.
- **Advantages:** Modular development, simplified debugging.
- **Disadvantages:** Hard to define layers cleanly, and poor performance because each layer crossing introduces execution overhead.

2.3 Microkernel Structure

The microkernel architecture structures the OS by removing all non-essential components from the kernel, implementing them instead as user-space programs (servers).

- **Core Kernel Services:** Low-level memory management, thread scheduling, and Inter-Process Communication (IPC).
- **Advantages:** High security and reliability. If a filesystem service crashes, the rest of the OS remains unaffected.
- **Disadvantages:** High performance overhead because client programs must communicate with user-space servers via message passing, which requires frequent context switches.

2.4 Hybrid Systems

Most modern operating systems (e.g., Windows NT, macOS, Linux) use a hybrid design. They combine monolithic speed with microkernel modularity, running core modules in a single address space while using dynamic modules (loadable kernel modules) for extensibility.

Quick Revision — Module 2

Operating System Structures | OE-EC604B

System Call Parameter Passing Methods

Register Method	Parameters passed directly in CPU registers. Fastest, but limited parameter capacity.
Block / Table Method	Parameters written to a block in memory; pointer to the block is passed in a register. Unlimited size.
Stack Method	Parameters pushed onto the user stack and popped off by the kernel. Unlimited size.

One-Line Definitions

- **System Call:** Programmatic request by user-mode applications to execute kernel-mode services.
- **Mode Bit:** A hardware register flag indicating User Mode (1) or Kernel Mode (0).
- **Monolithic Kernel:** OS design where all services run in a single address space inside kernel mode.
- **Microkernel:** Minimalist kernel executing only IPC, scheduling, and basic memory management.
- **Layered Architecture:** Hierarchy of OS layers where layer i relies only on services of layer $i - 1$.
- **Trap:** A software-generated interrupt caused by an error or system call request.
- **API (Application Programming Interface):** Standard set of functions wrapper around system calls (e.g., Win32, POSIX).
- **System Program:** File utilities, compilers, or editors that provide a shell wrapper around system calls.

Frequently Asked / Viva Questions

Q1. Why is dual-mode operation supported by hardware in modern operating systems?

To protect the system from malicious or poorly written user applications. Privileged instructions (like I/O control or memory protection access) can only be executed in Kernel Mode.

Q2. Describe the sequence of actions that occur when a system call is made.

The user program invokes the API function → API puts system call number in a register → triggers software interrupt (trap) → hardware switches mode bit to 0 → execution jumps to system call handler in kernel → handler executes kernel

Frequently Asked / Viva Questions (contd.)

code → mode bit is reset to 1 → control returns to user program.

Q3. Compare Monolithic Kernels and Microkernels in terms of reliability and performance.

Monolithic: High performance (fast function calls, no IPC overhead) but low reliability (any driver crash can bring down the entire OS). Microkernel: High reliability (isolated user-space services) but slower performance (heavy IPC message passing and context switches).

Q4. What are system programs? Give two examples.

System programs are user-space utilities that provide a convenient environment for program development and execution. Examples: file command programs (`cp`, `mv`) and status utilities (`ps`, `top`).

Q5. What are the three common methods used to pass parameters to the operating system during a system call?

1. Pass parameters in CPU registers. 2. Store parameters in a memory block/table and pass the block address in a register. 3. Push parameters onto the system stack.

Q6. What is the difference between a system call and a library function?

A system call is handled directly by the kernel and involves context switching and privilege escalation. A library function (like `strlen()`) executes entirely in user mode without kernel intervention.

Q7. Explain the concept of device drivers in OS structure.

Device drivers are specialized software modules that translate general I/O commands from the OS kernel into device-specific commands that can be interpreted by hardware controllers.

Q8. Why does the layered architecture model suffer from performance issues?

Each layer boundary requires crossing and parameter validation. An application requesting a file read might have to traverse multiple layers sequentially, incurring cumulative overhead.

Q9. What is the POSIX standard?

Portable Operating System Interface; a family of standards specified by the IEEE to clarify API compatibility between UNIX-like operating systems.

Q10. How does a hybrid operating system achieve both reliability and speed?

By using a monolithic core for speed (running speed-critical elements like graphics and file systems in kernel space) but supporting dynamic loadable kernel modules (LKMs) for driver modularity.

Comparison of OS Architectures

Architecture	Context Switch Cost	Reliability	Extensibility
Monolithic	Extremely Low (no IPC)	Poor (no protection)	Hard (requires recompile)
Layered	High (traverses layers)	Moderate	Moderate
Microkernel	Very High (heavy IPC)	Excellent	Very Easy (add user servers)

Module 3

Process Concept and Lifecycle States

1. Process Concept and Lifecycle States

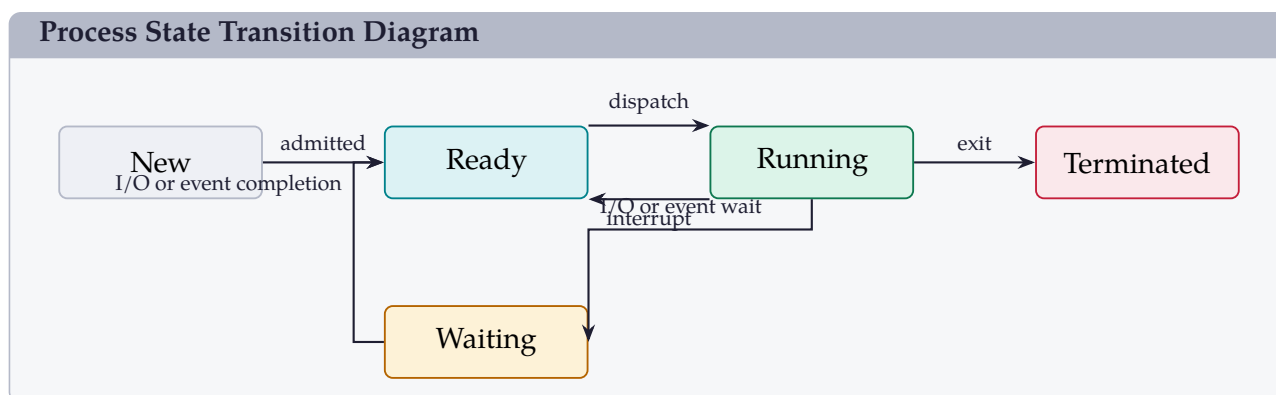
A **process** is a program in execution. It represents the unit of work in a modern time-sharing system.

1.1 Process Control Block (PCB)

Each process is represented in the operating system by a **Process Control Block (PCB)** (also called a task control block). The PCB contains:

- **Process State:** New, ready, running, waiting, or terminated.
- **Program Counter (PC):** Address of the next instruction to execute.
- **CPU Registers:** Accumulators, index registers, stack pointers.
- **CPU-Scheduling Information:** Process priority, pointers to scheduling queues.
- **Memory-Management Information:** Page tables or segment tables.
- **Accounting Information:** CPU time used, time limits.
- **I/O Status Information:** List of open files, allocated I/O devices.

1.2 Process States and Transitions



2. Process Generation & Scheduling Queues

2.1 Process Generation (`fork()` and `exec()`)

- **`fork()` System Call:** Creates a new, child process that is a duplicate of the parent process.
 - In the parent process, `fork()` returns the PID of the child.
 - In the child process, `fork()` returns 0.
 - If the call fails, it returns -1.
- **`exec()` System Call:** Replaces the process's memory space with a new executable program, changing the active program counter.

2.2 OS Schedulers

- **Long-Term Scheduler (Job Scheduler):** Selects processes from the pool (disk) and loads them into memory for execution. Controls the **degree of multiprogramming**.
- **Short-Term Scheduler (CPU Scheduler):** Selects from the processes in memory that are ready to execute and allocates the CPU to one of them. Executes extremely fast.
- **Medium-Term Scheduler:** Temporarily removes processes from memory (swapping) to reduce the degree of multiprogramming and free up RAM.

3. Inter-Process Communication (IPC)

Processes executing concurrently may be independent or cooperating. Cooperating processes require **Inter-Process Communication (IPC)**: 1. **Shared Memory:** A region of memory is established that cooperating processes can read and write to. Fast, but requires manual synchronization to prevent race conditions. 2. **Message Passing:** Processes exchange messages via the OS kernel without sharing memory space. Easier to implement but slower due to system calls.

4. Process Synchronization and Critical Section

A **race condition** occurs when multiple processes access and manipulate the same data concurrently, and the outcome of the execution depends on the particular order in which the access takes place.

4.1 The Critical Section Problem

The **Critical Section** is a code segment in which a process accesses shared resources.

Three Requirements for a Critical Section Solution

1. **Mutual Exclusion:** If process P_i is executing in its critical section, no other processes can execute in their critical sections.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter, only those processes not executing in their remainder sections can participate in deciding which process enters next.
3. **Bounded Waiting:** There must be a limit on the number of times other processes can enter their critical sections after a process has made a request to enter, before that request

is granted.

4.2 Semaphores

A **Semaphore** is an integer variable S that, apart from initialization, is accessed only through two standard atomic operations: `wait()` (or P) and `signal()` (or V).

Semaphore Atomic Implementations

```
void wait(Semaphore S) {
    while (S <= 0); // Busy wait (or block process in list)
    S--;
}

void signal(Semaphore S) {
    S++;
}
```

- **Binary Semaphore (Mutex):** Value can be either 0 or 1. Used for mutual exclusion.
- **Counting Semaphore:** Value ranges over an unrestricted integer domain. Used for resource allocation.

4.3 Classical Synchronization Problems

4.3.1 Bounded-Buffer (Producer-Consumer) Problem

Uses semaphores to coordinate a shared buffer of size N :

- `mutex` (binary, initial = 1): Protects the buffer insertion/removal.
- `empty` (counting, initial = N): Tracks empty buffer slots.
- `full` (counting, initial = 0): Tracks full buffer slots.

4.3.2 Readers-Writers Problem

Ensures multiple readers can access data concurrently, but only one writer can access it exclusively:

- `rw_mutex` (binary, initial = 1): Protects critical resource for writers.
- `mutex` (binary, initial = 1): Protects modification of `read_count` variable.

4.3.3 Dining Philosophers Problem

Represents resource allocation challenges among concurrent processes. 5 philosophers sit around a table with 5 chopsticks. Each needs 2 chopsticks to eat. Prone to deadlock if everyone picks up their left chopstick simultaneously.

4.4 Monitors

A **Monitor** is an abstract data type that encapsulates private data and procedures, ensuring that only one process can execute inside the monitor at any given time, preventing synchronization errors automatically.

Quick Revision — Module 3

Concurrent Processes | OE-EC604B

Producer-Consumer Semaphore Code

Producer Process:

```
do {
    // produce item
    wait(empty);
    wait(mutex);
    // add item to buffer
    signal(mutex);
    signal(full);
} while (true);
```

Consumer Process:

```
do {
    wait(full);
    wait(mutex);
    // remove item from buffer
    signal(mutex);
    signal(empty);
    // consume item
} while (true);
```

One-Line Definitions

- **Process:** A program in execution containing program counter, stack, data, and registers.
- **PCB:** Process Control Block; kernel data structure storing all attributes of a process.
- **Context Switch:** Saving CPU state of a process and restoring another to switch execution.
- **Race Condition:** State where multiple processes write/read shared data and outcome depends on execution order.
- **Critical Section:** Part of program accessing shared resource that must be protected.
- **Mutual Exclusion:** Preventing more than one process from entering a critical section.
- **Semaphore:** Protected integer variable used for signaling and process synchronization.
- **Monitors:** High-level programming construct providing automated mutual exclusion.

Frequently Asked / Viva Questions

Q1. Draw and label the 5-state process lifecycle model.

States: New → Ready → Running → Waiting → Terminated. Transitions: admitted, scheduler dispatch, interrupt, exit, I/O wait, I/O completion. (See TikZ block diagram).

Q2. What is the difference between fork() and exec() system calls?

Frequently Asked / Viva Questions (contd.)

`fork()` creates a new duplicate child process with its own PID, copying the parent's memory layout. `exec()` replaces the current process's entire memory space and code with a new binary program.

Q3. Describe the differences between Long-term, Medium-term, and Short-term schedulers.

Long-term: Controls degree of multiprogramming by loading jobs from disk to ready queue (RAM). Short-term: Allocates CPU to a ready process. Medium-term: Performs swapping of processes out of RAM to disk.

Q4. Define a race condition and explain how it can be prevented.

A race condition is when multiple processes access and write shared data concurrently and execution order dictates the final value. Prevented by enforcing mutual exclusion using synchronization mechanisms (semaphores, mutex).

Q5. Explain the three criteria that must be satisfied by any solution to the Critical Section problem.

1. Mutual Exclusion (only one process inside). 2. Progress (non-critical processes cannot block waiting ones). 3. Bounded Waiting (no starvation, waiting requests must be granted eventually).

Q6. Differentiate between Binary Semaphores and Counting Semaphores.

Binary semaphores take values 0 and 1 only, used primarily for mutual exclusion. Counting semaphores take any non-negative integer values, used to manage access to resources with multiple instances.

Q7. What is busy waiting? How do semaphores resolve it?

Busy waiting is when a process loops continuously checking a condition (`while(S <= 0);`), wasting CPU cycles. Resolved by having the semaphore block the process, putting it in a waiting queue and waking it up on `signal()`.

Q8. Describe the Dining Philosophers synchronization challenge.

5 philosophers share 5 chopsticks. If each picks up their left chopstick first, they will all wait indefinitely for their right chopstick, resulting in a deadlock.

Q9. What is a Monitor, and what advantage does it have over raw semaphores?

A monitor is a software package encapsulating shared variables and access methods. It ensures mutual exclusion automatically at compile-time/run-time, whereas semaphores are prone to programmer errors (like forgetting a `signal()` call).

Q10. Why does a context switch introduce performance overhead?

Because the CPU must save all register values, program counter, and memory maps of the current process into its PCB, and load those of the new process, during which no useful computation is done.

IPC Models Comparison

Feature	Shared Memory	Message Passing
Data Transfer Speed	Extremely Fast (RAM speeds)	Slower (system call overhead)
Kernel Involvement	Only during setup	For every message exchange
Synchronization	Managed by programmer (mutex)	Handled by OS kernel implicitly
Use Case	Large data volume, single machine	Small data volume, distributed systems

Module 4

Scheduling Concepts and Performance Criteria

1. Scheduling Concepts and Performance Criteria

In a single-processor system, only one process can run at a time; others must wait until the CPU is free. The objective of multiprogramming is to run multiple processes concurrently to maximize CPU utilization.

1.1 CPU-I/O Burst Cycle

Process execution consists of a cycle of **CPU execution (CPU burst)** and **I/O wait (I/O burst)**. Processes alternate between these two states.

- **CPU-bound Process:** Spends most of its time doing computations (long CPU bursts).
- **I/O-bound Process:** Spends most of its time waiting for I/O (short CPU bursts).

1.2 Scheduling Criteria

Operating systems select processes from the ready queue based on five key performance criteria: 1. **CPU Utilization:** The percentage of time the CPU is busy executing tasks (Goal: Maximize, 40% to 90%). 2. **Throughput:** The number of processes completed per unit time (Goal: Maximize). 3. **Turnaround Time (TAT):** The total interval from process submission to its completion (Goal: Minimize). 4. **Waiting Time (WT):** The total time a process spends waiting in the ready queue (Goal: Minimize). 5. **Response Time (RT):** The time from submission to the first CPU response (Goal: Minimize, critical for interactive systems).

2. CPU Scheduling Algorithms

CPU scheduling decisions take place under four conditions: 1. When a process switches from running to waiting (Non-Preemptive). 2. When a process switches from running to ready (Preemptive). 3. When a process switches from waiting to ready (Preemptive). 4. When a process terminates (Non-Preemptive).

2.1 First-Come, First-Served (FCFS)

- **Principle:** The process that requests the CPU first gets it first. Managed via a FIFO queue.
- **Drawback:** Non-preemptive. Suffers from the **Convoy Effect** — short processes must wait for a single long process to finish, causing high average waiting times.

2.2 Shortest Job First (SJF)

- **Principle:** Associates with each process the length of its next CPU burst. The process with the shortest burst is scheduled.
- **SJF (Non-preemptive):** Once CPU is allocated, it cannot be preempted.
- **SRTF (Preemptive SJF):** If a new process arrives with a shorter remaining burst than the currently running process, the CPU is preempted.
- **Advantage:** SJF is **optimal** — it gives the minimum average waiting time for a given set of processes.
- **Disadvantage:** Cannot be implemented directly because the length of the next CPU burst is unknown. Prone to **starvation** of long processes.

2.3 Round Robin (RR) Scheduling

- **Principle:** Designed for time-sharing systems. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval up to **1 time quantum (slice)** (q).
- **Working:** If process burst $> q$, it is preempted and put back at the tail of the ready queue.
- **Performance Dependency:** If q is extremely large, RR behaves like FCFS. If q is extremely small, it causes high context-switching overhead.

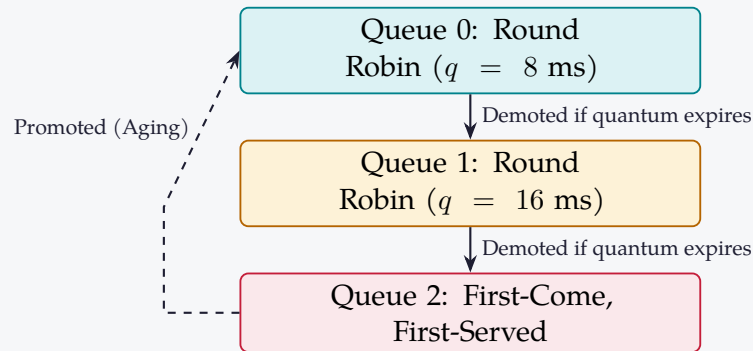
2.4 Priority Scheduling

- **Principle:** A priority number is associated with each process, and the CPU is allocated to the process with the highest priority (lowest integer usually represents highest priority).
- **Problem: Starvation (Indefinite Blocking)** of low-priority processes.
- **Solution: Aging** — gradually increasing the priority of processes that wait in the system for a long time.

2.5 Multilevel Queue (MLQ) & Multilevel Feedback Queue (MLFQ)

- **MLQ:** Partitioned ready queue into several separate queues (e.g., foreground interactive and background batch) with fixed priority. No movement between queues.
- **MLFQ:** Allows processes to move between queues based on CPU burst history. If a process uses too much CPU time, it is demoted. If it waits too long, it is promoted (aging).

Multilevel Feedback Queue Structure



3. Multiprocessor Scheduling

When multiple CPUs are available, scheduling complexity increases:

- **Asymmetric Multiprocessing:** One master processor handles all scheduling decisions, I/O processing, and system activities; others run only user code. Simple, but master is a bottleneck.
- **Symmetric Multiprocessing (SMP):** Each processor schedules itself from the common ready queue or private ready queue.
- **Processor Affinity:** A process has an affinity for the processor it is currently running on to avoid the cache invalidation cost of migrating to another CPU.
 - **Soft Affinity:** OS tries to keep process on same CPU but does not guarantee it.
 - **Hard Affinity:** Process specifies it can only run on a subset of CPUs.
- **Load Balancing:** Keeps workload evenly distributed across all CPUs.
 - **Push Migration:** A specific task monitors load and pushes processes from overloaded CPUs to idle ones.
 - **Pull Migration:** An idle processor pulls a waiting task from a busy processor.

Quick Revision — Module 4

CPU Scheduling | OE-EC604B

Core Scheduling Formulas

Turnaround Time (TAT) = Completion Time (CT) – Arrival Time (AT)

Waiting Time (WT) = Turnaround Time (TAT) – Burst Time (BT)

Response Time (RT) = First CPU Start Time – Arrival Time (AT)

$$\text{Average Waiting Time} = \frac{1}{N} \sum_{i=1}^N WT_i$$

One-Line Definitions

- **Throughput:** Number of completed processes per unit time.
- **Convoy Effect:** Scenario in FCFS where short jobs wait behind a very long job, degrading latency.
- **Preemptive Scheduling:** OS mechanism to interrupt a running process and assign CPU to another.
- **Quantum (q):** The fixed time slice allocated to a process in Round Robin scheduling.
- **Starvation:** Situation where a process waits indefinitely because other processes have higher priorities.
- **Aging:** Technique of gradually increasing the priority of waiting processes to prevent starvation.
- **Processor Affinity:** Cache optimization keeping a process executing on the same CPU core.
- **Symmetric Multiprocessing (SMP):** Multiprocessing where each CPU makes its own scheduling decisions.

Frequently Asked / Viva Questions

Q1. Compare FCFS, SJF, and Round Robin scheduling algorithms.

FCFS is simple, non-preemptive, but prone to convoy effect. SJF is optimal for average waiting time but causes starvation and is hard to predict. Round Robin is preemptive, quantum-based, and ideal for interactive systems.

Q2. Show the Gantt chart and compute Turnaround and Waiting times for:

Frequently Asked / Viva Questions (contd.)

Process	Arrival Time	Burst Time	Priority
P1	0	8	3 (Low)
P2	1	4	1 (High)
P3	2	2	2

Using Preemptive Priority Scheduling:

Gantt Chart: P1 runs from [0-1]. At $t = 1$, P2 arrives with priority 1, preempts P1. P2 runs from [1-5]. At $t = 5$, P3 runs from [5-7]. At $t = 7$, P1 finishes from [7-14].

Completion Times (CT): P2 = 5, P3 = 7, P1 = 14.

Turnaround Times (TAT = CT - AT): P1 = 14, P2 = 4, P3 = 5. (Avg TAT = 7.67).

Waiting Times (WT = TAT - BT): P1 = 6, P2 = 0, P3 = 3. (Avg WT = 3.0).

Q3. What is the Convoy Effect in CPU scheduling? How is it resolved?

It happens in FCFS when one CPU-bound process holds the CPU while multiple I/O-bound processes wait in the ready queue. Resolved by using preemptive scheduling algorithms (like Round Robin).

Q4. Why is SJF scheduling called "optimal"? Why is it difficult to implement?

It is optimal because it mathematically minimizes the average waiting time for a set of processes. It is hard to implement because the exact burst time of future processes cannot be known in advance (it must be estimated using exponential smoothing).

Q5. How does the performance of Round Robin scheduling depend on the time quantum?

If the quantum is too large, it degenerates into FCFS. If it is too small, it causes excessive context-switching overhead, which wastes CPU cycles. A rule of thumb is that 80% of CPU bursts should be shorter than the quantum.

Q6. What is starvation? How does aging solve it?

Starvation occurs when low-priority processes never get scheduled because high-priority processes keep arriving. Aging solves this by gradually increasing the priority of a process the longer it remains waiting in the ready queue.

Q7. Differentiate between Multilevel Queue (MLQ) and Multilevel Feedback Queue (MLFQ).

In MLQ, processes are permanently assigned to a queue based on type, with no queue switching. In MLFQ, processes can move dynamically between queues based on CPU behavior, enabling automatic detection of I/O vs CPU bound jobs.

Q8. Explain Processor Affinity and its types.

Processor affinity is the policy of keeping a process on the same CPU to leverage cached data. Soft affinity means the OS tries to enforce this but can migrate the process. Hard affinity guarantees the process runs only on designated CPUs.

Frequently Asked / Viva Questions (contd.)

Q9. What is the difference between Push and Pull migration in load balancing?

Push migration runs a task that periodically checks load and "pushes" processes from busy CPUs to idle ones. Pull migration occurs when an idle CPU queries other CPUs and "pulls" an waiting process to execute.

Q10. Explain CPU-I/O burst cycle.

A process's lifecycle alternates between CPU execution blocks (CPU bursts) and waiting for device I/O actions (I/O bursts), ending with a final CPU burst and termination request.

CPU Scheduling Algorithms Comparison

Algorithm	Preemptive?	Starvation Risk?	Optimal average WT?
FCFS	No	No	No
SJF	No	Yes	Yes (among non-preemptive)
SRTF	Yes	Yes	Yes (optimal)
Round Robin	Yes	No	No
Priority	Yes/No	Yes	No

Module 5

System Model and Deadlock Characterization

1. System Model and Deadlock Characterization

A **deadlock** is a state where a set of processes are blocked because each process is holding a resource and waiting for another resource held by some other process in the set.

1.1 Four Necessary Conditions (Coffman Conditions)

A deadlock situation can arise if and only if all four of the following conditions hold simultaneously in a system: 1. **Mutual Exclusion:** At least one resource must be held in a non-shareable mode (only one process can use it at a time). 2. **Hold and Wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes. 3. **No Preemption:** Resources cannot be preempted; a resource can be released only voluntarily by the process holding it, after that process has finished its task. 4. **Circular Wait:** A closed loop of processes must exist, where each process waits for a resource held by the next process in the loop. E.g., P_0 waits for R_1 (held by P_1), P_1 waits for R_2 (held by P_2), and P_n waits for R_0 (held by P_0).

1.2 Resource Allocation Graph (RAG)

Deadlocks can be described precisely using a directed graph called a **Resource Allocation Graph (RAG)**:

- **Vertices (V):** Divided into two partitions:
 - Processes $P = \{P_1, P_2, \dots, P_n\}$ (represented by circles).
 - Resource types $R = \{R_1, R_2, \dots, R_m\}$ (represented by rectangles containing dots for instances).
- **Edges (E):**
 - **Request Edge:** Directed edge $P_i \rightarrow R_j$ (process P_i has requested resource R_j).
 - **Assignment Edge:** Directed edge $R_j \rightarrow P_i$ (resource instance in R_j is allocated to process P_i).
- **Theorem:**
 - If the RAG contains no cycles $\rightarrow$ No deadlock.
 - If the RAG contains a cycle $\rightarrow$
 - If single instance per resource type: Deadlock exists (cycle is necessary and sufficient).

- If multiple instances per resource type: Deadlock *may* exist (cycle is necessary but not sufficient).

2. Deadlock Handling Strategies

Operating systems use three main strategies to handle deadlocks:

2.1 1. Deadlock Prevention

Ensures that the system can never enter a deadlock state by designing the system such that at least one of the four Coffman conditions cannot hold:

- **Prevent Mutual Exclusion:** Make resources shareable (e.g., read-only files). **Drawback:** Some resources (printers, mutexes) are inherently non-shareable.
- **Prevent Hold and Wait:** Require processes to request all resources at once before execution, or release current resources before requesting new ones. **Drawback:** Low resource utilization; starvation.
- **Prevent No Preemption:** If a process holding resources requests another that cannot be immediately allocated, all its current resources are preempted (released). **Drawback:** Overhead of saving/restoring process states.
- **Prevent Circular Wait:** Impose a global ordering on all resource types (e.g., $F(R_j) = i$) and require processes to request resources only in strictly increasing order of enumeration. **Advantage:** Most practical prevention strategy.

2.2 2. Deadlock Avoidance

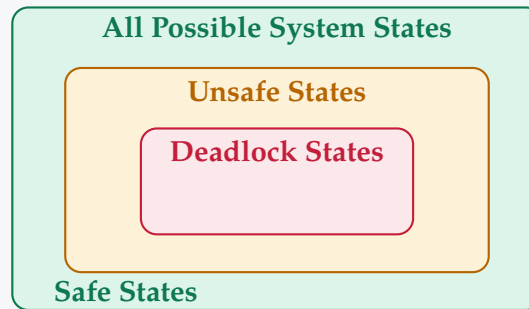
Requires that the OS has prior knowledge of the maximum resources a process will ever request.

Safe State vs Unsafe State

A state is **safe** if there exists a **safe sequence** $\langle P_1, P_2, \dots, P_n \rangle$ of execution such that, for each P_i , the resources that P_i can still request can be satisfied by the currently available resources plus the resources held by all prior processes P_j (for $j < i$).

- **Relationship:** A safe state is not deadlocked. An unsafe state is not necessarily deadlocked, but it can lead to a deadlock if processes request their maximum limits. Avoidance algorithms keep the system inside a safe state.

Deadlock State Space Partitioning



2.2.1 Banker's Algorithm

Used for systems with multiple instances of resources. Data structures:

- $Available[m]$: Available instances of each resource.
- $Max[n][m]$: Maximum demand of each process.
- $Allocation[n][m]$: Resources currently allocated to each process.
- $Need[n][m]$: Remaining resources a process may request. Note: $Need[i][j] = Max[i][j] - Allocation[i][j]$.

2.3 3. Deadlock Detection & Recovery

If a system does not prevent or avoid deadlocks, a deadlock may occur. The OS must provide:

- A detection algorithm (periodically runs to find cycles or execute safety checks).
- A recovery scheme:
 - **Process Termination**: Abort all deadlocked processes, or abort one-by-one until the cycle is broken.
 - **Resource Preemption**: Preempt resources from some processes, roll back those processes to a safe checkpoint, and restart them.

Quick Revision — Module 5

Deadlocks | OE-EC604B

Banker's Safety Algorithm Equations

Let $Work = Available$ and $Finish[i] = false$ for $i = 0, 1, \dots, n - 1$. Find an index i such that both: 1. $Finish[i] == false$ 2. $Need_i \leq Work$ If such an i exists, update:

$$Work = Work + Allocation_i$$

$$Finish[i] = true$$

Repeat. If $Finish[i] == true$ for all i , the system is in a **Safe State**.

One-Line Definitions

- **Deadlock:** State where processes wait indefinitely for resources held by each other.
- **Mutual Exclusion:** Rule that only one process can hold a resource instance at any time.
- **Hold and Wait:** Process holding resources while requesting more without releasing current ones.
- **Preemption:** Forcible retrieval of allocated resources from a process by the OS.
- **Circular Wait:** Chain of processes where each waits for a resource held by the next.
- **Resource Allocation Graph:** Directed graph showing resource allocations and process requests.
- **Safe Sequence:** Order of process execution that guarantees no deadlock can occur.
- **Ostrich Algorithm:** Deadlock strategy of ignoring the problem (used when deadlocks are rare).

Frequently Asked / Viva Questions

Q1. What are the four necessary conditions for a deadlock to occur?

Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait (Coffman conditions). All four must hold simultaneously for a deadlock to exist.

Q2. Consider a system with 5 processes (P0 to P4) and 3 resource types (A, B, C):

Frequently Asked / Viva Questions (contd.)

$$\text{Allocation} = \begin{pmatrix} 0 & 1 & 0 \\ 2 & 0 & 0 \\ 3 & 0 & 2 \\ 2 & 1 & 1 \\ 0 & 0 & 2 \end{pmatrix}, \quad \text{Max} = \begin{pmatrix} 7 & 5 & 3 \\ 3 & 2 & 2 \\ 9 & 0 & 2 \\ 2 & 2 & 2 \\ 4 & 3 & 3 \end{pmatrix}, \quad \text{Available} = (3 \ 3 \ 2)$$

Is the system in a safe state? Find the safe sequence.

Step 1: Calculate Need matrix (Need = Max – Allocation)

$$\text{Need} = \begin{pmatrix} 7 - 0 & 5 - 1 & 3 - 0 \\ 3 - 2 & 2 - 0 & 2 - 0 \\ 9 - 3 & 0 - 0 & 2 - 2 \\ 2 - 2 & 2 - 1 & 2 - 1 \\ 4 - 0 & 3 - 0 & 3 - 2 \end{pmatrix} = \begin{pmatrix} 7 & 4 & 3 \\ 1 & 2 & 2 \\ 6 & 0 & 0 \\ 0 & 1 & 1 \\ 4 & 3 & 1 \end{pmatrix}$$

Step 2: Initialize Work = (3 3 2)

- **Check P1:** Need₁ (1 2 2) ≤ Work (3 3 2)? Yes. Work → Work + Alloc₁ = (5 3 2).
- **Check P3:** Need₃ (0 1 1) ≤ Work (5 3 2)? Yes. Work → Work + Alloc₃ = (7 4 3).
- **Check P0:** Need₀ (7 4 3) ≤ Work (7 4 3)? Yes. Work → Work + Alloc₀ = (7 5 3).
- **Check P2:** Need₂ (6 0 0) ≤ Work (7 5 3)? Yes. Work → Work + Alloc₂ = (10 5 5).
- **Check P4:** Need₄ (4 3 1) ≤ Work (10 5 5)? Yes. Work → Work + Alloc₄ = (10 5 7).

Result: All processes finished. System is in **safe state** with safe sequence: <P1, P3, P0, P2, P4>.

Q3. Differentiate between Deadlock Prevention and Deadlock Avoidance.

Prevention limits system design by enforcing policies that violate one of the 4 Coffman conditions (e.g., resource ordering). Avoidance does not restrict request structures but checks dynamic resource availability and maximum limits to keep the system in a safe state.

Q4. How can the circular wait condition be prevented in operating systems?

Frequently Asked / Viva Questions (contd.)

By defining a global 1-to-1 enumeration function $F : R \rightarrow \mathbb{N}$ for all resource types. A process can only request resource R_j if $F(R_j) > F(R_i)$, where R_i is any resource currently held by the process.

Q5. What is an unsafe state? Does it always lead to a deadlock?

An unsafe state is a state from which the OS cannot guarantee that all processes can finish their maximum execution. It does **not** always lead to a deadlock; it will only deadlock if processes actually request their maximum declared resources.

Q6. Why does a cycle in a Resource Allocation Graph (RAG) not always indicate a deadlock?

If a resource type has multiple instances, a cycle indicates a potential deadlock but not a guaranteed one, as a process outside the cycle might release an instance, breaking the cycle.

Q7. What is the difference between a Resource Allocation Graph and a Wait-For Graph?

A Resource Allocation Graph shows both processes and resources with request/assignment edges. A Wait-For Graph is a simplified version for single-unit resource systems, showing only processes, where edge $P_i \rightarrow P_j$ means P_i waits for P_j to release a resource.

Q8. Explain the recovery methods from deadlock.

1. Process Termination: Abort all deadlocked processes, or abort processes one-by-one until the cycle is broken. 2. Resource Preemption: Take resources from a process, roll it back to a safe checkpoint, and restart it.

Q9. What is the "Ostrich Algorithm" and when is it appropriate to use?

It is the strategy of ignoring deadlocks altogether, assuming they occur rarely. It is used by most general-purpose OS (like Windows/Linux) because deadlock prevention/avoidance adds heavy run-time overhead.

Q10. Why is preemption difficult to implement for some resources?

Because some resources (like printers or files being written) cannot be easily saved and restored mid-operation without corrupting data or output.

Deadlock Handling Trade-offs

Strategy	Run-time Overhead	Resource Utilization	Implementation
Prevention	None (static rules)	Low (strict limits)	Simple
Avoidance	Moderate (safety checks)	Moderate	Complex (needs max demand)
Detection	High (periodic checks)	High (free allocation)	Moderate

Module 6

Contiguous Memory Allocation

1. Contiguous Memory Allocation

Main memory must accommodate both the operating system and various user processes. In contiguous memory allocation, each process is contained in a single contiguous section of memory.

1.1 Memory Partitioning Schemes

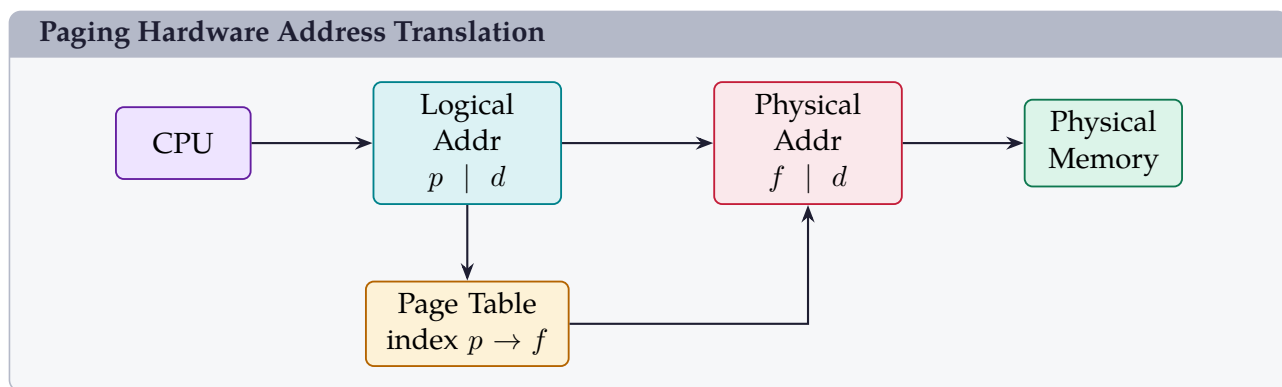
- **Fixed Partitioning (MFT - Multiprogramming with a Fixed number of Tasks):** Memory is divided into fixed-size partitions. Each partition contains exactly one process.
 - **Drawback:** Leads to **Internal Fragmentation** — if a process is smaller than the partition allocated to it, the remainder space is wasted.
- **Variable Partitioning (MVT - Multiprogramming with a Variable number of Tasks):** OS maintains a table of free memory slots (holes). When a process arrives, it is allocated a hole matching its size.
 - **Drawback:** Leads to **External Fragmentation** — total free memory is large enough to satisfy a request, but it is not contiguous (split into small holes).
 - **Solution: Compaction** — shifting all processes to one end of memory to group all free holes into a single block (possible only if dynamic relocation is used).

2. Non-Contiguous Memory Allocation

To eliminate external fragmentation and the need for compaction, modern systems use non-contiguous memory allocation.

2.1 Paging Architecture

- **Frames:** Physical memory is broken into fixed-size blocks (typically 4 KB).
- **Pages:** Logical memory is broken into blocks of the same size.
- **Address Translation:** Every address generated by the CPU is divided into:
 - **Page Number (p):** Used as an index into a **Page Table** to find the base physical frame.
 - **Page Offset (d):** Combined with the base frame address to yield the physical address.



- **TLB (Translation Lookaside Buffer):** A fast hardware cache of associative registers used to store recent page-to-frame translations. It reduces the memory access penalty from two cycles (one for page table, one for data) to close to one cycle.

2.2 Segmentation

Segmentation is a memory-management scheme that supports the programmer's view of memory. A logical address space is a collection of variable-length segments (e.g., main program, procedure, stack, variables).

- **Segment Table:** Maps logical address $\langle \text{segment_number}, \text{offset} \rangle$ to physical addresses. Each entry has a **base** (start physical address) and **limit** (length of segment).

3. Virtual Memory and Demand Paging

Virtual Memory allows execution of processes that are not completely in main memory. It enables extremely large logical address spaces.

3.1 Demand Paging & Page Faults

Pages are loaded into RAM only when they are referenced during execution (lazy swapper).

- **Page Table Entry:** Includes a **valid-invalid bit** (V/I). V means page is in memory; I means page is on disk.
- **Page Fault:** Occurs when a running process references a page marked as invalid.
- **Page Fault Handling Sequence:**

1. Trap to OS on page fault.
2. Save process registers and state.
3. Verify reference is valid, find page location on disk.
4. Read page from disk into a free frame (using a page replacement algorithm if no free frames exist).
5. Update page table (set bit to V).
6. Restart the instruction that was interrupted by the trap.

3.2 Page Replacement Algorithms

When no frames are free, the OS must swap a page out to make room.

- **FIFO (First-In, First-Out):** Replaces the oldest page. Prone to **Belady's Anomaly** — page fault rate increases as physical frame allocation increases.

- **Optimal (OPT):** Replaces the page that will not be used for the longest time in the future. Optimal benchmark, but impossible to implement (requires future knowledge).
- **LRU (Least Recently Used):** Replaces the page that has not been referenced for the longest time in the past. Requires hardware support (counter or stack).

3.3 Thrashing

Thrashing is a state where a process spends more time paging (swapping pages in and out of disk) than executing useful instructions.

- **Cause:** The process has been allocated fewer frames than its **working set** (the set of pages it is actively using). As a result, it page faults immediately, forcing page swaps in a continuous loop.
- **Solution:** Working-Set Model (allocates frames based on active page locality) or Page-Fault Frequency (PFF) control.

Quick Revision — Module 6

Memory Management | OE-EC604B

Virtual Memory Math

$$\text{Effective Access Time (EAT)} = (1 - p) \times T_{\text{mem}} + p \times T_{\text{fault}}$$

where p = Page Fault Rate ($0 \leq p \leq 1$), T_{mem} = RAM Access Time, T_{fault} = Page Fault Service Time (disk seek + read).

One-Line Definitions

- **Internal Fragmentation:** Wasted memory inside a fixed partition allocated to a smaller process.
- **External Fragmentation:** Free memory blocks distributed non-contiguously, preventing allocation.
- **Paging:** Non-contiguous allocation breaking physical memory into fixed frames and logical into pages.
- **TLB:** Translation Lookaside Buffer; hardware cache to speed up logical-to-physical address mapping.
- **Segmentation:** Memory allocation method based on logical units (code, stack, global data).
- **Demand Paging:** Lazy page allocation technique loading pages only on execution references.
- **Belady's Anomaly:** Phenomenon where allocating more physical frames increases page faults in FIFO.
- **Thrashing:** System state where CPUs sit idle because processes are locked in page-swap loops.

Frequently Asked / Viva Questions

Q1. Compare Paging and Segmentation.

Paging divides memory into fixed-size blocks (pages/frames), avoiding external fragmentation but suffering from internal fragmentation. It is invisible to the user. Segmentation divides memory into logical variable-size modules, matching the programmer's view, but suffers from external fragmentation.

Q2. Given a reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3. Frame size = 3.

Calculate page faults for: (a) FIFO, (b) LRU, (c) Optimal.

(a) FIFO: Frames status:

- Ref 7: [7] - Fault 1

Frequently Asked / Viva Questions (contd.)

- **Ref 0:** [7, 0] - Fault 2
- **Ref 1:** [7, 0, 1] - Fault 3
- **Ref 2:** [2, 0, 1] (replaces 7) - Fault 4
- **Ref 0:** [2, 0, 1] - Hit
- **Ref 3:** [2, 3, 1] (replaces 0) - Fault 5
- **Ref 0:** [2, 3, 0] (replaces 1) - Fault 6
- **Ref 4:** [4, 3, 0] (replaces 2) - Fault 7
- **Ref 2:** [4, 2, 0] (replaces 3) - Fault 8
- **Ref 3:** [4, 2, 3] (replaces 0) - Fault 9. **Total Faults = 9.**

(b) **LRU:** Frames status:

- **Ref 7, 0, 1:** [7, 0, 1] - Faults = 3
- **Ref 2:** [2, 0, 1] (replaces 7) - Fault 4
- **Ref 0:** [2, 0, 1] - Hit (0 is most recent)
- **Ref 3:** [2, 0, 3] (replaces 1) - Fault 5
- **Ref 0:** [2, 0, 3] - Hit
- **Ref 4:** [4, 0, 3] (replaces 2) - Fault 6
- **Ref 2:** [4, 0, 2] (replaces 3) - Fault 7
- **Ref 3:** [3, 0, 2] (replaces 4) - Fault 8. **Total Faults = 8.**

(c) **Optimal:** Frames status:

- **Ref 7, 0, 1:** [7, 0, 1] - Faults = 3
- **Ref 2:** [2, 0, 1] (replaces 7, unused furthest) - Fault 4
- **Ref 0:** [2, 0, 1] - Hit
- **Ref 3:** [2, 0, 3] (replaces 1, unused furthest) - Fault 5
- **Ref 0:** [2, 0, 3] - Hit
- **Ref 4:** [4, 0, 3] (replaces 2, unused furthest) - Fault 6
- **Ref 2:** [2, 0, 3] (replaces 4) - Fault 7
- **Ref 3:** [2, 0, 3] - Hit. **Total Faults = 7.**

Q3. What is Belady's Anomaly? Give an example of when it occurs.

Belady's Anomaly is when allocating more page frames to a process increases the number of page faults. It occurs in FIFO page replacement. Example string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. Frame size = 3 gives 9 faults; Frame size = 4 gives 10 faults.

Q4. What is a TLB and how does it improve Effective Access Time?

A TLB is a fast associative cache on the CPU containing translation table entries. Instead of doing two memory accesses (one for page table in RAM, one for target location), the CPU checks the TLB. If it's a hit, the translation is resolved in < 1 cycle, minimizing EAT.

Frequently Asked / Viva Questions (contd.)

Q5. Explain the concept of compaction in memory management.

Compaction is a technique used to resolve external fragmentation in variable partitioning. The OS dynamically shifts active processes in memory to make them contiguous, merging all small free holes into a single large free partition.

Q6. What is thrashing? How can it be detected and prevented?

Thrashing occurs when a process is constantly swapping pages in and out of disk because it has been allocated fewer frames than its active working set. Detected when CPU utilization drops and disk queuing spikes. Prevented using the Working-Set Model or Page-Fault Frequency controls.

Q7. Describe the page fault handling mechanism.

Process references page → MMU detects invalid bit → traps to OS kernel → OS checks virtual address and finds page on disk → allocates free frame → schedules I/O to read page → updates page table to valid → resumes the process.

Q8. How does logical address space differ from physical address space?

Logical address space is the set of all virtual addresses generated by a program during compilation/execution. Physical address space is the set of actual physical hardware addresses in main memory (RAM).

Q9. Explain the difference between internal and external fragmentation.

Internal fragmentation occurs when allocated memory is slightly larger than the process's demand, leaving wasted space inside the partition. External fragmentation occurs when free memory is sufficient but broken into small, non-contiguous blocks.

Q10. What is a dirty bit (modify bit) and how does it optimize page replacement?

A dirty bit is a flag set in the page table entry when a page is written to. During page replacement, if the dirty bit is 0 (page unchanged), the OS can overwrite the frame without writing it back to disk, halving page-fault overhead.

Page Replacement Algorithms summary

Algorithm	Implementation Complexity	Hardware Support?	Belady's Anomaly?
FIFO	Very Low	None	Yes
Optimal	Impossible (needs future data)	None	No
LRU	High (requires stacks/counters)	Yes (clock/reference bits)	No

Module 7

I/O Hardware and Organization of I/O Functions

1. I/O Hardware and Organization of I/O Functions

The control of I/O devices connected to the computer is a major function of an operating system. The OS must issue commands to devices, catch interrupts, and handle errors.

1.1 I/O Communication Methods

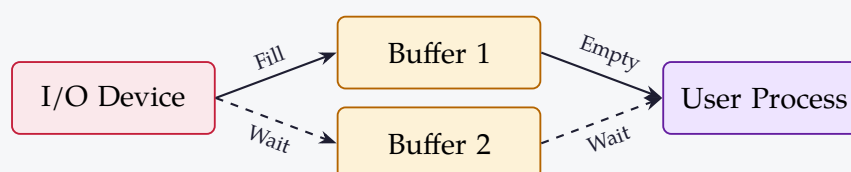
- **Polling (Busy-Wait I/O):** CPU repeatedly reads the status register of the device controller until the device is ready. **Drawback:** Wastes CPU cycles.
- **Interrupt-Driven I/O:** CPU starts I/O and switches to other tasks. When I/O is ready, the device controller triggers a hardware interrupt, forcing the CPU to branch to the Interrupt Service Routine (ISR).
- **Direct Memory Access (DMA):** Used for high-speed I/O. The DMA controller transfers blocks of data directly between device buffer and main memory, interrupting the CPU only once at the end of the block transfer.

1.2 I/O Buffering

A buffer is a memory area that stores data temporarily while it is being transferred between two devices or between a device and an application.

- **Single Buffering:** OS assigns a buffer in system memory. The I/O device writes to this buffer, and then the OS copies it to user space.
- **Double Buffering (Ping-Ponging):** OS assigns two buffers. While the user process consumes data from buffer 1, the device writes to buffer 2 in parallel.
- **Circular Buffering:** A queue of three or more buffers used when bursty transfer rates fluctuate.

Double Buffering Model



2. Disk Physical Structure and Access Performance

Magnetic disks provide the bulk of secondary storage for modern systems.

- **Structure:** Disks consist of platters coated with magnetic material. Each platter surface is divided into concentric circles called **tracks**, which are split into small sectors (typically 512 Bytes). Corresponding tracks on all platters form a **cylinder**.
- **Disk Access Time:**
 - **Seek Time (T_s):** Time required to move the disk arm to the target cylinder. (Dominates disk latency).
 - **Rotational Latency (T_r):** Time for the target sector to rotate under the read-write head. (Avg latency = $\frac{1}{2} \times$ time for one rotation).
 - **Transfer Time (T_t):** Time to actually read/write data. $T_t = \frac{b}{r \cdot N}$ where b = bytes to transfer, r = rotation speed, N = bytes per track.

3. Disk Scheduling Algorithms

Because seek time dominates access latency, the OS schedules incoming disk requests to minimize head travel distance.

Request Queue Example

For the algorithms below, assume a disk with 200 cylinders (0–199). The request queue is: 98, 183, 37, 122, 14, 124, 65, 67. Read-write head starts at cylinder 53.

3.1 First-Come, First-Served (FCFS)

- **Working:** Services requests in the order of arrival.
- **Head Movement Path:** 53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67.
- **Total head cylinders traversed:** 640. Fair, but very inefficient.

3.2 Shortest Seek Time First (SSTF)

- **Working:** Selects the request closest to the current head position.
- **Head Movement Path:** 53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183.
- **Total head cylinders traversed:** 236. High efficiency but prone to **starvation** of outer cylinders.

3.3 SCAN (Elevator Algorithm)

- **Working:** Arm starts at one end, moves toward the other end, servicing requests, until it reaches the boundary (cylinder 0 or 199), then reverses direction.
- **Head Movement Path (moving towards 0):** 53 → 37 → 14 → 0 → 65 → 67 → 98 → 122 → 124 → 183.

- **Total head cylinders traversed:** 236. Prevents starvation, but treats boundary requests unfairly.

3.4 C-SCAN (Circular SCAN)

- **Working:** Moves from one end to the other servicing requests. When it reaches the end, it immediately returns to the starting boundary without servicing any requests on the return trip.
- **Head Movement Path (moving towards 199):** 53 → 65 → 67 → 98 → 122 → 124 → 183 → **199** → **0** → 14 → 37.
- **Total head cylinders traversed:** 382. Provides more uniform waiting times.

3.5 LOOK & C-LOOK

- **Working:** Similar to SCAN and C-SCAN but the arm goes only as far as the **last request** in each direction, rather than traveling to the extreme boundaries (0 or 199).
- **C-LOOK Path (moving towards 183):** 53 → 65 → 67 → 98 → 122 → 124 → 183 → **14** → 37.
- **Total head cylinders traversed:** 322. Saves travel time compared to C-SCAN.

Quick Revision — Module 7

I/O Management and Disk Scheduling | OE-EC604B

Disk Parameter Definitions

Total Disk Access Time = Seek Time + Rotational Latency + Transfer Time

$$\text{Average Rotational Latency} = \frac{1}{2} \times \frac{60}{\text{RPM}} \text{ seconds}$$

One-Line Definitions

- **DMA:** Direct Memory Access; I/O technique transferring data blocks to memory without CPU overhead.
- **Double Buffering:** OS buffering model using two registers in parallel to remove consumption stalls.
- **Seek Time:** Time taken for the read-write head assembly to align with the target cylinder.
- **Rotational Latency:** Time taken for the target sector to rotate under the active head.
- **SSTF:** Shortest Seek Time First; scheduling that selects request with minimal seek distance.
- **SCAN:** Elevator scheduling algorithm traversing the entire disk width in cycles.
- **C-SCAN:** Circular SCAN; sweeps in one direction, returning to start instantly without servicing.
- **C-LOOK:** SCAN variant that returns immediately after servicing the last request in that direction.

Frequently Asked / Viva Questions

Q1. Explain the difference between Polling, Interrupt-driven I/O, and DMA.

Polling has the CPU busy-wait checking status registers. Interrupt-driven I/O triggers hardware interrupts when data is ready, letting the CPU execute other tasks. DMA uses a dedicated controller to transfer large blocks directly between device and memory, interrupting the CPU only once at block completion.

Q2. Compare SCAN, C-SCAN, LOOK, and C-LOOK disk scheduling algorithms.

SCAN moves the head to the extreme boundary in one direction, then reverses. C-SCAN goes to the boundary, returns to the other boundary without servicing on the return. LOOK and C-LOOK improve on this by reversing at the last request instead of going to the extreme boundary.

Q3. Give a numerical comparison of SCAN vs LOOK head travel. Head starts at 50 (direction: increasing). Request queue: 90, 150, 20. Boundary = 199.

Frequently Asked / Viva Questions (contd.)

SCAN: Head goes $50 \rightarrow 90 \rightarrow 150 \rightarrow 199 \rightarrow 20$. Head travel: $(199 - 50) + (199 - 20) = 149 + 179 = 328$ cylinders.

LOOK: Head goes $50 \rightarrow 90 \rightarrow 150 \rightarrow 20$ (stops at 150 because it is the last request). Head travel: $(150 - 50) + (150 - 20) = 100 + 130 = 230$ cylinders.

Q4. What is starvation in disk scheduling? Which algorithm is most prone to it?

Starvation occurs when a request is never serviced because closer requests keep arriving. SSTF is most prone to starvation because the head tends to stay in localized clusters, leaving requests on the far ends unserved.

Q5. Explain the mechanical layout of a Hard Disk Drive.

A spindle holds multiple circular platters. Read-write heads are mounted on an actuator arm that moves radially. Each platter surface contains concentric tracks, and tracks are divided into sectors. Corresponding tracks vertically form cylinders.

Q6. What is double buffering and why is it preferred over single buffering?

Double buffering assigns two buffers. While the CPU reads/empties buffer 1, the device controller fills buffer 2 concurrently. In single buffering, the CPU must stall while the single buffer is being re-filled.

Q7. Define Seek Time, Rotational Latency, and Transfer Time.

Seek time is the time to align the arm over the cylinder. Rotational latency is the time for the sector to rotate to the head. Transfer time is the actual data read/write rate.

Q8. A disk rotates at 7200 RPM. What is its average rotational latency?

7200 rotations in 60 seconds $\rightarrow$ 1 rotation takes $60/7200 = 8.33$ ms. Average rotational latency is half of one rotation time: $0.5 \times 8.33 = 4.17$ ms.

Q9. Why does C-SCAN offer more uniform waiting times than SCAN?

In SCAN, cylinders near the middle are swept twice as often as those near the boundaries. C-SCAN sweeps in one direction only, returning to start instantly, giving all cylinders a uniform wait interval between sweeps.

Q10. What are the main design issues in I/O system design?

Achieving device independence (programs can write to any device without custom code), uniform naming, error handling, buffering optimization, and asynchronous operation.

Disk Scheduling Algorithms Comparison

Algorithm	Average Seek Time	Starvation Risk?	Boundary Travel?
FCFS	High (non-optimal)	No	No
SSTF	Low (highly localized)	Yes	No
SCAN	Low	No	Yes (goes to end)
C-LOOK	Low to Moderate	No	No (reverses at last request)

Module 8

File and Directory Concepts

1. File and Directory Concepts

The file system provides the online storage and access mechanism for both data and programs of the operating system.

1.1 File Concepts & Access Mechanisms

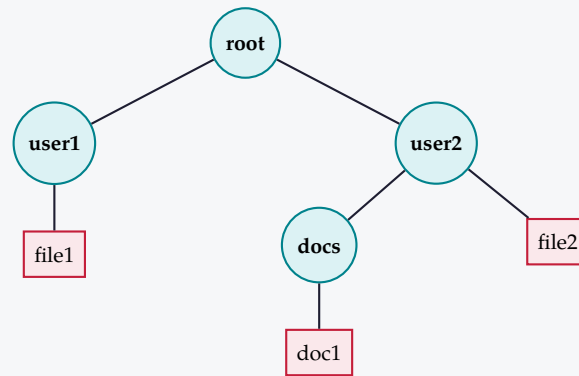
- **File Concept:** A file is a named collection of related information that is recorded on secondary storage.
- **Access Methods:**
 - **Sequential Access:** Information is processed in order, one record after another (e.g., tape storage).
 - **Direct (Random) Access:** A file is viewed as a numbered sequence of blocks. The program can read/write any block instantly in any order (e.g., database queries on disk).
 - **Indexed Access:** An index table is built containing pointers to various blocks. The program searches the index first to retrieve records.

1.2 Directory Structures

A directory is a container that organizes files. Common topologies:

- **Single-Level Directory:** All files are in a single directory. **Problems:** Naming collisions, organization issues.
- **Two-Level Directory:** Each user gets a private User File Directory (UFD) nested under a Master File Directory (MFD).
- **Tree-Structured Directory:** Users can create subdirectories and organize files hierarchically. The most common structure.

Tree-Structured Directory Tree



2. File System Implementation and Allocation Methods

To store files on disk, the OS must map logical file blocks to physical disk sectors. There are three traditional allocation methods:

2.1 1. Contiguous Allocation

- **Working:** Each file occupies a contiguous set of blocks on the disk.
- **Advantages:** Fast sequential read/write operations because the disk head does not need to move. Easy direct access.
- **Disadvantages:** Prone to **external fragmentation**. Hard to grow a file dynamically if adjacent blocks are already allocated.

2.2 2. Linked Allocation

- **Working:** Each file is a linked list of disk blocks. Directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block.
- **Advantages:** No external fragmentation. Files can grow indefinitely.
- **Disadvantages:** Slow random (direct) access (must traverse links sequentially from block 1). Pointers consume storage space. If a pointer gets corrupted, the rest of the file is lost.
- **FAT (File Allocation Table):** A variation of linked allocation. Pointers are stored in a centralized table at the beginning of the volume, caching entries for faster lookup.

2.3 3. Indexed Allocation

- **Working:** Each file has its own **index block**, which contains an array of pointers to all its physical data blocks.
- **Advantages:** Supports direct access without traversing lists. No external fragmentation.
- **Disadvantages:** Index block overhead — wastes space for very small files. If a file grows too large, a single index block is insufficient. (Resolved using linked index blocks, multilevel indexing, or UNIX indirect blocks).

3. Free Space Management

To allocate blocks, the OS tracks all free disk blocks using one of two common methods:

1. **Bit Vector (Bit Map):** An array of bits where each bit corresponds to a disk block (0 = allocated, 1 = free). **Advantages:** Fast, easy to find consecutive blocks. **Disadvantages:** Must be kept in RAM, consumes space for large disks.
2. **Linked List:** All free blocks are linked together. Directory keeps a pointer to the first free block. **Advantages:** Minimal space overhead. **Disadvantages:** Slow to traverse; hard to find contiguous free blocks.

Quick Revision — Module 8

File Systems | OE-EC604B

File Allocation Calculations

Let block size = 1 KB. Pointer size = 4 Bytes.

- **Single Index Block Limit:** An index block can hold $1 \text{ KB} / 4 \text{ B} = 256$ pointers. Max file size = $256 \times 1 \text{ KB} = 256 \text{ KB}$.
- **Two-Level Indexing Limit:** Primary index block points to 256 secondary index blocks. Max file size = $256 \times 256 \times 1 \text{ KB} = 64 \text{ MB}$.

One-Line Definitions

- **File:** A named logical collection of related records stored on secondary storage.
- **Sequential Access:** Access method where data is read in sequence from beginning to end.
- **Direct Access:** Random block access where records are accessed directly by block index.
- **Contiguous Allocation:** Allocation method where file blocks are saved in consecutive disk sectors.
- **Linked Allocation:** Allocation method where blocks are linked via pointers in a list structure.
- **FAT:** File Allocation Table; centralized directory indexing system caching linked pointers.
- **Indexed Allocation:** Allocation method where all block pointers are collected in an index block.
- **Bit Vector:** Binary sequence where each bit represents the free/allocated state of a disk block.

Frequently Asked / Viva Questions

Q1. Compare Contiguous, Linked, and Indexed file allocation methods.

Contiguous is fast, supports direct access, but suffers from external fragmentation and file-growth limitations. Linked eliminates fragmentation and allows files to grow easily but has slow random access and pointer overhead. Indexed supports direct access and file growth with no fragmentation, but has index block space overhead.

Q2. Explain the FAT (File Allocation Table) file system structure.

FAT is a variation of linked allocation. It extracts all block pointers from the data blocks and stores them in a centralized table at the beginning of the partition. The directory entry contains the starting block number, and the table is indexed

Frequently Asked / Viva Questions (contd.)

by block numbers to find subsequent blocks.

Q3. What is an index block in file system implementation? What happens if a file exceeds its capacity?

An index block contains an array of disk block addresses allocated to a file. If a file exceeds the index block capacity, the OS uses: 1. Linked scheme (linking multiple index blocks). 2. Multilevel indexing (an index block containing pointers to other index blocks). 3. Combined scheme (like UNIX inodes).

Q4. Describe free space management using a Bit Vector. Calculate the size of a bit map for a 200 GB disk with 4 KB blocks.

A bit vector uses 1 bit for each disk block: 1 means free, 0 means allocated. **Total Blocks** = $200 \text{ GB} / 4 \text{ KB} = (200 \times 10^9) / (4 \times 10^3) = 50 \times 10^6$ blocks.

Map Size = 50×10^6 bits ≈ 6.25 MB of storage.

Q5. Explain the difference between sequential and direct file access mechanisms.

Sequential access processes records one after another from the beginning, moving a read pointer sequentially. Direct access allows reading/writing blocks in any arbitrary order by specifying the relative block number directly.

Q6. What are the advantages of a tree-structured directory over a single-level directory?

A tree-structured directory allows users to create nested subdirectories to organize files logically, preventing naming conflicts and enabling file organization, sharing, and access control.

Q7. Explain the implementation issues of contiguous allocation.

Finding space for a new file requires executing dynamic allocation algorithms (first-fit, best-fit), which leads to external fragmentation. Also, if a file grows, it may collide with an adjacent allocated file, requiring compaction.

Q8. Why does linked allocation suffer from reliability issues?

Because the blocks are scattered across the disk, and each block contains a pointer to the next. If a single sector gets corrupted, the pointer is lost, rendering all subsequent blocks inaccessible.

Q9. What is the purpose of directory mounting?

Mounting allows the OS to attach multiple physical file systems on different storage devices (hard disks, USB drives) into a single, unified logical directory tree.

Q10. How does the OS optimize directory lookup speed?

By using hash tables or B-trees instead of linear lists to store directory entries, reducing lookup times from $O(N)$ to $O(1)$ or $O(\log N)$.

File Allocation Methods Comparison

Feature	Contiguous	Linked	Indexed
Direct Access	Excellent	Very Poor (linear traversal)	Excellent
External Frag.	Yes	None	None
Storage Efficiency	High	Moderate (pointer overhead)	Low for small files
File Growth	Hard (contiguous block)	Easy	Easy

Module 9

Protection & Security

1. Protection & Security

Modern computer systems must protect data from unauthorized access, modification, and destruction.

- **Protection:** A mechanism for controlling the access of programs, processes, or users to the resources defined by a computer system.
- **Security:** A measure of confidence that the integrity of a system and its data will be preserved against external threats (viruses, hackers, physical damage).

1.1 Access Matrix Model

The **Access Matrix** is a formal protection model. It views the system as a set of **domains** (active entities, e.g., users, processes) and **objects** (passive hardware/software components, e.g., files, printers).

- **Grid Entry:** $\text{Matrix}[D, O]$ defines the access rights (read, write, execute) that process in domain D has over object O .
- **Implementations:**
 - **Access Control List (ACL):** Associates with each object a list of domains and their permitted rights (columns of the matrix). Used in Unix/Windows filesystems.
 - **Capability List (C-List):** Associates with each domain a list of objects and rights (rows of the matrix).

2. Introduction to Distributed Operating Systems

A **Distributed Operating System** manages a collection of independent, networked, and physically separate computational nodes, presenting them to the user as a single, unified computer system.

- **Network OS vs. Distributed OS:** In a Network OS, users are aware of the existence of multiple computers (e.g., must use `ssh` or remote mounting). A Distributed OS provides **location transparency** — the user cannot tell where a program is running or where a file is stored.

3. Case Study: The UNIX Operating System

UNIX is a highly modular, multitasking operating system originally developed at Bell Labs.

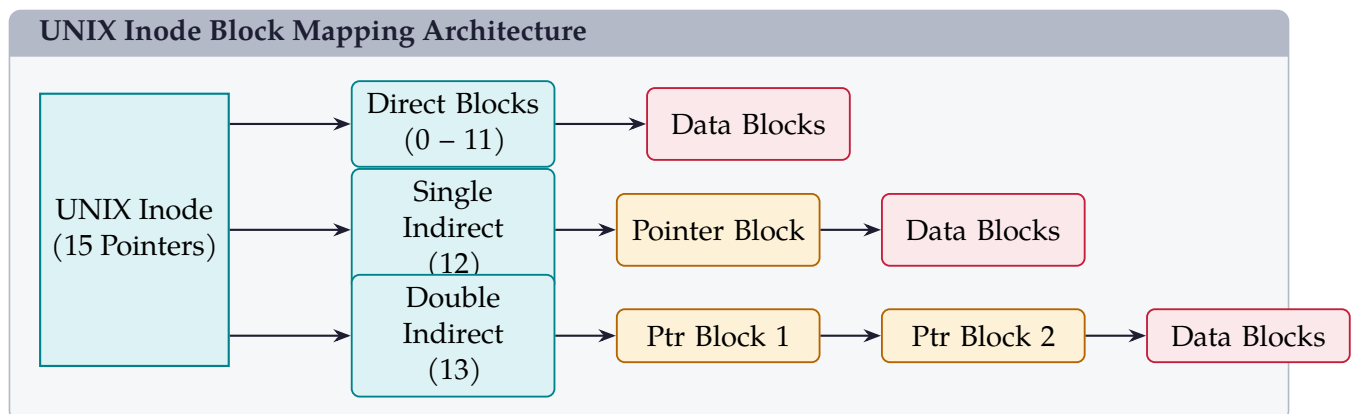
3.1 System Architecture

UNIX is divided into three main layers: 1. **User Space:** Applications, library calls, and the command interpreter (Shell). 2. **Kernel Space:** Manages process execution, memory, file systems, interrupts, and I/O. 3. **Hardware:** The underlying CPU, memory, and physical controllers.

3.2 The UNIX File System (Inode Structure)

In UNIX, every file is represented by a data structure called an **Inode (Index Node)**. The inode contains file metadata (owner, permissions, size, time stamps) and a table of **block addresses** pointing to the physical data on disk.

- **Inode Address Pointers Layout:** Typically contains 15 pointers:
 - **Pointers 0–11: Direct Blocks** (point directly to disk data blocks).
 - **Pointer 12: Single Indirect Block** (points to a block containing pointers to data blocks).
 - **Pointer 13: Double Indirect Block** (points to a block containing pointers to single indirect blocks).
 - **Pointer 14: Triple Indirect Block** (points to a block containing pointers to double indirect blocks).



3.3 UNIX Process Management & Scheduling

- **Process Creation:** Handled exclusively via `fork()` followed by `exec()`.
- **UNIX Process States:** Running in user mode, Running in kernel mode, Ready to run in memory, Asleep in memory (waiting for I/O), Ready to run swapped, Asleep swapped, Zombie (terminated but parent has not read exit status).
- **UNIX CPU Scheduling:** Uses a multilevel feedback queue scheduling algorithm. Priority is recalculated every second based on CPU usage:

$$\text{Priority} = \text{Base} + \frac{\text{Recent CPU Usage}}{2} + \text{Nice}$$

where `Nice` is a user-controllable process parameter (higher `Nice` value lowers priority).

Quick Revision — Module 9

Protection, Security and UNIX Case Study | OE-EC604B

UNIX Inode Max File Size Calculations

Let block size = 1 KB. Disk address pointer = 4 Bytes.

- **Max Direct Blocks Capacity:** $12 \times 1 \text{ KB} = 12 \text{ KB}$.
- **Max Single Indirect Capacity:** $(1 \text{ KB}/4 \text{ B}) \times 1 \text{ KB} = 256 \times 1 \text{ KB} = 256 \text{ KB}$.
- **Max Double Indirect Capacity:** $256 \times 256 \times 1 \text{ KB} = 64 \text{ MB}$.
- **Max Triple Indirect Capacity:** $256 \times 256 \times 256 \times 1 \text{ KB} = 16 \text{ GB}$.
- **Total Max File Size Limit:** $12 \text{ KB} + 256 \text{ KB} + 64 \text{ MB} + 16 \text{ GB} \approx 16.06 \text{ GB}$.

One-Line Definitions

- **Access Matrix:** Grid model representing domains (rows), objects (columns), and access rights (cells).
- **ACL:** Access Control List; column-wise access matrix stored on objects indicating permitted domains.
- **Capability List:** Row-wise access matrix stored inside process domains detailing object rights.
- **Distributed OS:** OS presenting multiple physical CPU nodes as a single logical computer transparently.
- **Location Transparency:** File or computation access method independent of physical node location.
- **Superblock:** UNIX filesystem metadata block containing system size, free block count, and inode limits.
- **UNIX Inode:** Index block detailing metadata and 15 storage pointers for a UNIX file.
- **Zombie Process:** Process that has terminated but still occupies an entry in the process table.

Frequently Asked / Viva Questions

Q1. Differentiate between Protection and Security in operating systems.

Protection is the internal OS mechanism to control process/user access to defined system resources. Security is the external defense mechanism protecting system integrity and data against threats like malware, hackers, and environment disasters.

Q2. Compare Access Control Lists (ACLs) and Capability Lists (C-Lists).

ACLs are column-oriented implementations of the Access Matrix; they associate

Frequently Asked / Viva Questions (contd.)

permitted domains and rights directly with each object (e.g., file permissions). C-Lists are row-oriented; they attach object rights to each domain/user process directly.

Q3. What is a Distributed Operating System? What are its primary advantages?

A Distributed OS coordinates multiple autonomous CPUs connected by communication networks, presenting them as a single logical machine. Advantages include resource sharing, computational speedup, reliability (fault tolerance), and scalability.

Q4. Explain the UNIX Inode pointer layout and calculate max file size for a 2 KB block size with 4-byte pointers.

Layout has 12 direct, 1 single indirect, 1 double indirect, and 1 triple indirect pointers. **Pointers per block** = $2\text{ KB} / 4\text{ B} = 512$.

Direct size = $12 \times 2\text{ KB} = 24\text{ KB}$.

Single indirect size = $512 \times 2\text{ KB} = 1024\text{ KB} = 1\text{ MB}$.

Double indirect size = $512^2 \times 2\text{ KB} = 512\text{ MB}$.

Triple indirect size = $512^3 \times 2\text{ KB} = 256\text{ GB}$.

Max File Size = $24\text{ KB} + 1\text{ MB} + 512\text{ MB} + 256\text{ GB} \approx 256.5\text{ GB}$.

Q5. What is the function of the Superblock in the UNIX file system?

The Superblock is located at a fixed offset near the start of the disk partition. It contains critical metadata about the file system, including total blocks, free block count, free inode list, size of inode list, and status flag.

Q6. Describe the UNIX process scheduling formula and explain the role of the nice value.

Priority = Base + (Recent CPU Usage/2) + Nice. The OS decays priority based on CPU usage to prevent CPU starvation. The `Nice` value is an administrator/user offset (ranging from -20 to $+19$). A higher nice value increases the priority value, which actually lowers scheduling priority.

Q7. What is a zombie process in UNIX? How is it cleared?

A zombie process is a process that has finished execution but still has an entry in the system process table to allow its parent to read its exit code (via `wait()`). It is cleared when the parent executes a `wait()` system call. If the parent terminates, the zombie is inherited by the `init` process, which automatically calls `wait()`.

Q8. Differentiate between Loosely Coupled and Tightly Coupled distributed systems.

Loosely coupled systems have independent nodes with private memory connected via slow communication networks (e.g., LAN clusters). Tightly coupled systems share a common physical memory and bus architecture with fast inter-processor communication.

Frequently Asked / Viva Questions (contd.)

Q9. What is the Access Matrix model? How is it represented?

It is a security model where rows represent domains, columns represent objects, and cell entries represent the set of access rights (read, write, execute) allowed for a domain over an object.

Q10. Explain the role of the shell in the UNIX operating system.

The shell is a user-space application that acts as a command line interpreter, reading user commands, expanding wildcards, setting up I/O redirection/piping, and spawning processes using `fork/exec` to execute those commands.

Security Implementation Comparison

Feature	Access Control List (ACL)	Capability List (C-List)
Association	Attached to the Object (e.g. File)	Attached to the Subject (e.g. Domain)
Revocation	Easy (just delete domain from file list)	Hard (must search all domain lists)
Access check	Slow (must search file ACL at access)	Fast (domain presents token directly)
Common use	Standard OS filesystems	Object-oriented/microkernel systems